Computer Architecture and System Software

Lecture 10: Exceptional Control Flow

Instructor:
David Bosc
Applied Computer Science
University of Winnipeg

Announcements

- Important Dates:
  - Final Exam dates announced (Wednesday, April 15th, 6pm-9pm @ **1D62** (Duckworth Gym), details of content to come)
  - Assignment 4 Released
    - Due April 1st at midnight
    - 1 question, 20 points
  - Final Lab due this week
- SET Evaluations
- Final exam review will be part of next lecture

Student Evaluation of Teaching (SET)

- Please complete your SET (Student Evaluation of Teaching) survey for this course. Your feedback is very important and appreciated.
- Look for a survey invite in your UW student email address. Example: timshel-m@webmail.uwinnipeg.ca
- Note that the email may be in the "Other" folder rather than the "Focused" folder in your inbox.
- Surveys close on the last day of regularly scheduled classes at 11:59 p.m.

Processes and the OS

Background to Sect. 8.1

Processes and the OS

- What is an O/S?
  - Is a program (or programs)
  - Runs on the same CPU
  - Uses the same memory
  - Accesses the IO devices
  - Is the guardian of the computer

The Role of an Operating System

- What are its roles?
  - Manage and run all our programs
  - Allow programs to share the CPU
  - Manage computer resources
    - Processor, memory, hard disk, network connection, and I/O devices
  - Protect processes from each other
  - Present a standard abstraction to programs and users
  - Act as an intermediary between the programs, user, and computer

The Role of an Operating System

The O/S:

- Is the first program loaded by the computer
- Resides in memory all the time
- Runs when
  - A program needs a service such as:
    - performing I/O, run a program, stop running, and allocation of more resources
  - An external event occurs
  - A program error occurs
  - A computer error occurs

8

Exceptions

Section 8.1

Exceptions

- Recall what the processor does:
  - The processor executes one instruction after another
  - The PC points to the next instruction to execute
  - PC takes on a sequence of values a_0, a_1, a_2, …
  - Where each a_k is the address of a corresponding instruction I_k
  - Each transition from a_k to a_{k+1} is called a control transfer
    - CPU control is transferred from instruction at a_k to one at a_{k+1}
  - A sequence of control transfers is called the flow of control

Control Flow and Exceptional Control Flow

- Program control flow
  - Transfers are either smooth or abrupt
- Smooth: Instruction I_{k+1} is adjacent to instruction I_k in memory
- Abrupt: I_{k+1} is not adjacent to instruction I_k in memory
- Occurs during:
  - Jump or branch
  - Procedure call
  - Procedure return

Control Flow and Exceptional Control Flow

- Abrupt control flow transfer are necessary to:
  - Allow a program to respond to changes of internal states
  - Execute conditional code (if statements)
  - Execute repetitive code (loops)
  - Use methods and procedures
- This is what we mean when we talk of program control flow

Observations

- Program has illusion that it is the only thing running on the computer
- However, system must react to events outside of the program
  - Keystrokes
  - Internet packets
  - Mouse movement
  - Inter-program signals
- How is this done?

Exceptional Control Flow (ECF)

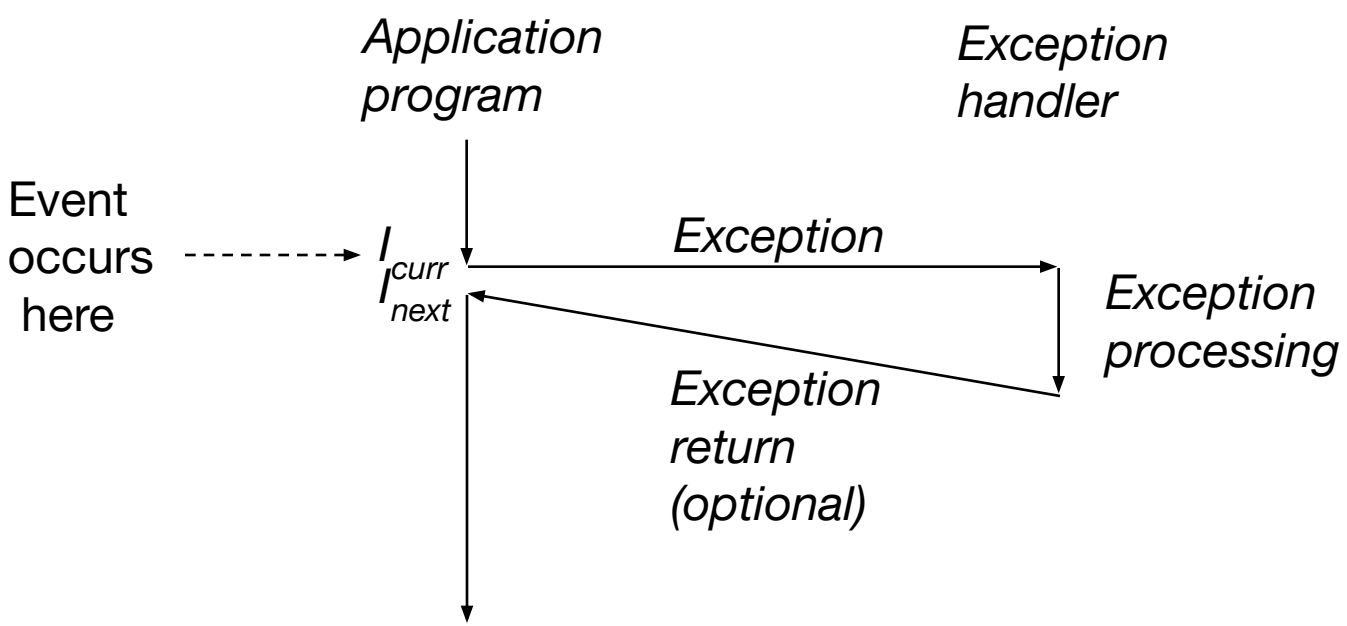
- System reacts to these events by special abrupt changes in control flow called **exceptional control flow (ECF)**
- These occur at all levels of a system
  - Hardware: devices can interrupt current CPU execution via exception handlers
  - Operating system: can transfer control between programs via context switches
  - Programs can abruptly change flow of other programs via signals and software exceptions

Exceptional Control Flow (ECF)

- We want to study ECF in order to understand how
  - An OS works (basic mechanism)
  - Programs (applications) interact with the OS
  - To write more interesting applications
  - Software exceptions work

Exceptions

- Exception is an abrupt change of flow control implemented jointly by hardware and OS
- When an exception occurs



Exceptions

- CPU is executing instruction I_{curr}
- A significant change in the processor's state occurs (called an event)
- Examples:
  - Divide by 0
  - Virtual memory page fault
  - Arithmetic overflow
  - I/O request completes
  - Hardware timer

Exceptions

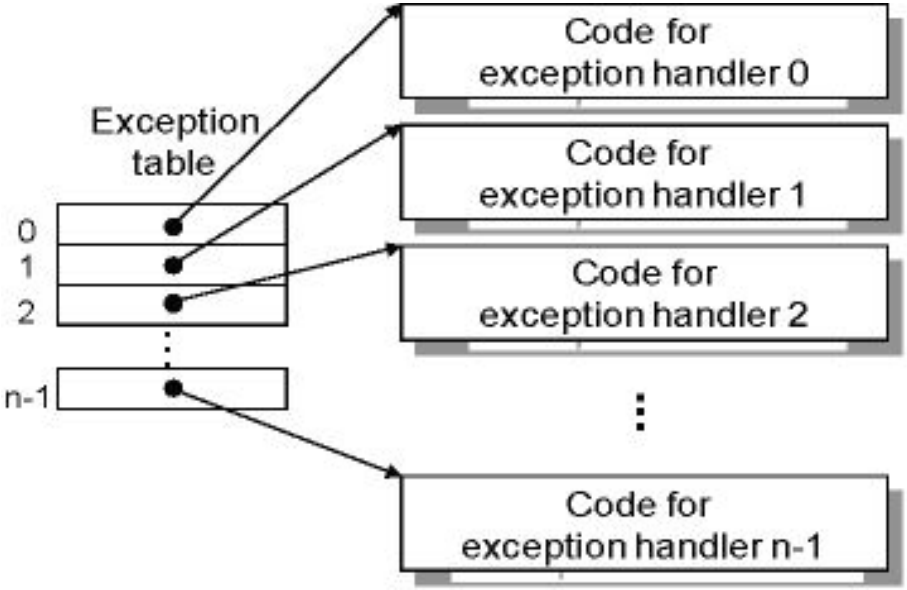
- Processor detects change
- Processor responds by making an indirect procedure call
  - This is why it is called an exception
- Procedure call is made through a lookup table called an exception table
- The procedure is an operating system subroutine called the exception handler
- After handler terminates, one of three things happen:
  - Handler returns control to instruction I_{curr}
  - Handler returns control to instruction I_{next}
  - Handler aborts program

Exception Handling

- Requires close cooperation between OS and CPU
- Each type of possible exception is assigned a unique nonnegative integer
  - Some values are determined by CPU designers
  - Others by kernel designers
  - There are a fixed number of possible exceptions
    - Pentium has 256
    - See, *e.g.*, http://stanislavs.org/helppc/int_table.html

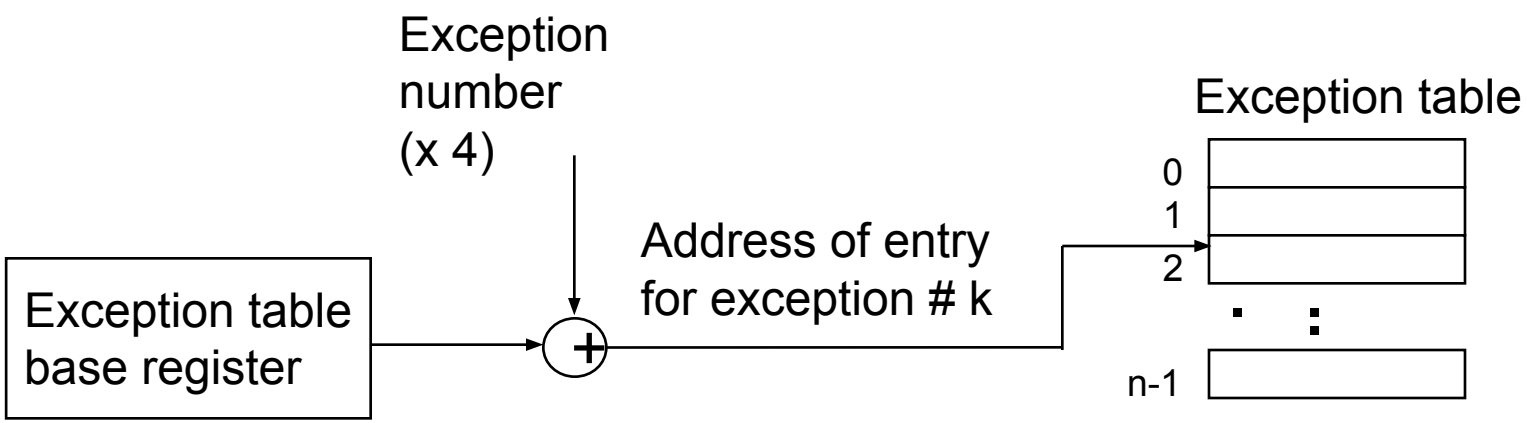
Exception Table

- Located in memory
- Created and initialized when OS boots
- Entry k contains the address of the handler for exception k
- Address of table is stored in CPU register called Exception Table Base Register
- CPU uses table to locate exception handler



Exception Table

- When exception occurs (CPU detects an event)
  - CPU determines ID of exception
  - Uses ID to index into exception table
  - Locates reference to corresponding handler
  - Uses reference to make a procedure call to handler



Observations

- This call differs from regular procedure call in several ways
  - Return address pushed on stack may be either current or next instruction, depending on type of exception
  - Processor also pushes additional information on stack
    - e.g. the flags register
  - Exception handler has complete control over the system
  - The handler then handles the event
  - What does the handler do ?
  - Depends on the type of exception

Class of Exceptions

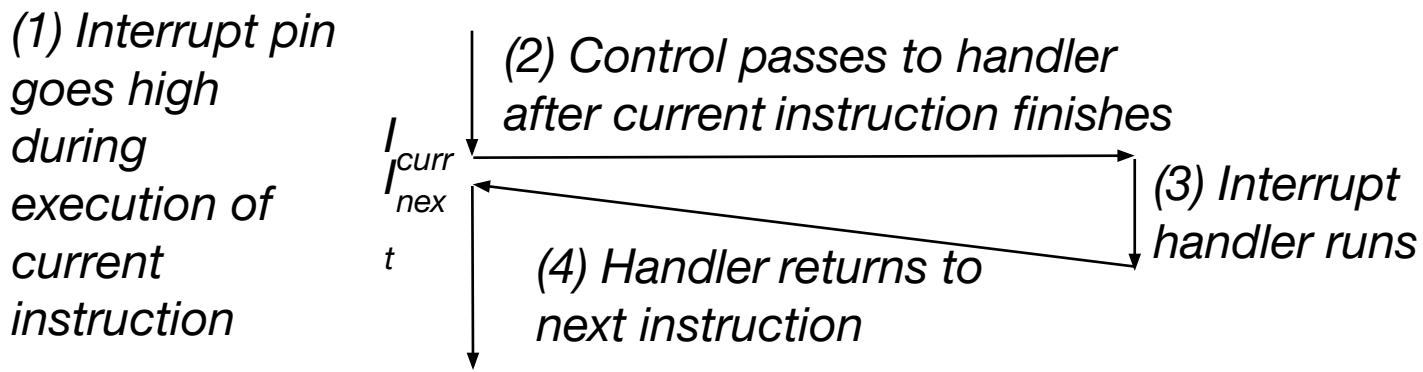
- There are four classes of exceptions:
  - Interrupts: caused by normal hardware events
  - Traps: caused by program service requests
  - Faults: caused by repairable program "errors"
  - Aborts: caused by irreparable program errors and HW errors
- Let's look at this table:
  - http://stanislavs.org/helppc/int_table.html

Interrupts

- Occur asynchronously, as a result of a hardware event
  - Mouse movements, keystrokes, I/O completions
  - Not caused by the execution of a instruction
- Handlers for interrupts are called interrupt handlers

Interrupt Handling

- A hardware devices signals a pin on the CPU
- At the same time, places exception ID on the system bus
- CPU finishes current instruction
- CPU reads ID from system bus
- Invokes handler to handle the device
- Handler completes
- Returns to the next instruction of program
- Program continues to execute as if interrupt did not occur



Class of Exceptions

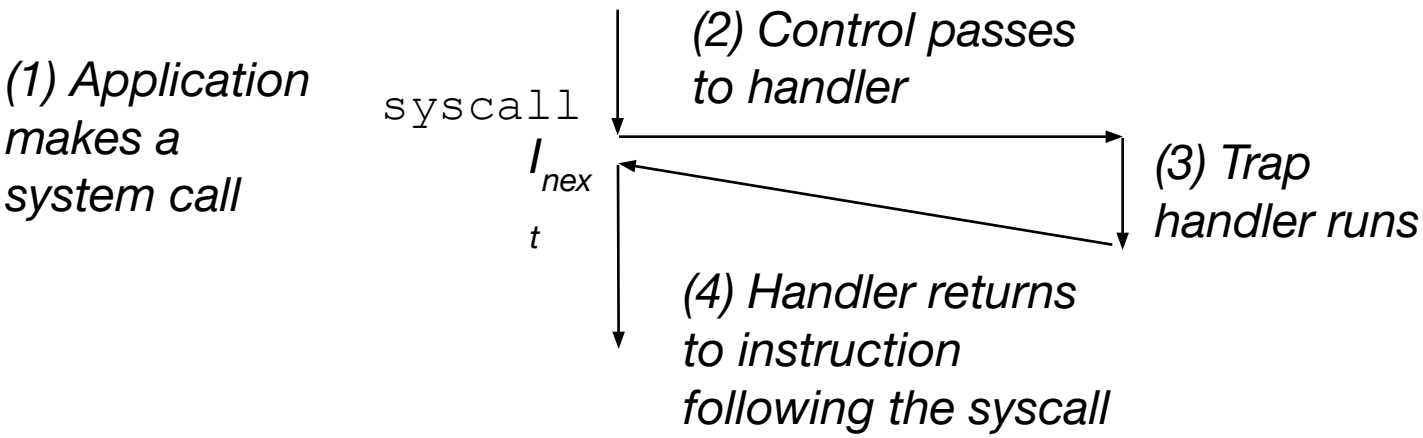
- The remaining exceptions are all synchronous
  - Traps: caused by program service requests
  - Faults: caused by repairable program “errors”
  - Aborts: caused by unrepairable program errors and HW errors
- Caused by the current instruction
- This instruction is called the faulting instruction

Traps

- Intentional exceptions (called a “system call”)
- Occur as a result of executing a special “trap” instruction
- Used by programs to request services from OS
  - Read/write data from/to a file
  - Create a new process (run program)
  - Terminate program
- Provides a procedure-like interface between OS and application
- From a programmer’s perspective call looks just like a function call, except
  - Regular functions run in “user” mode
  - System calls run in “kernel” mode

Trap Handling

- Program invokes "trap" instruction with ID as operand
- CPU uses ID to index into exception table, locate handler
- Handler is invoked to handle system call
- Handler completes
- Returns to next instruction of program (similar to interrupt)

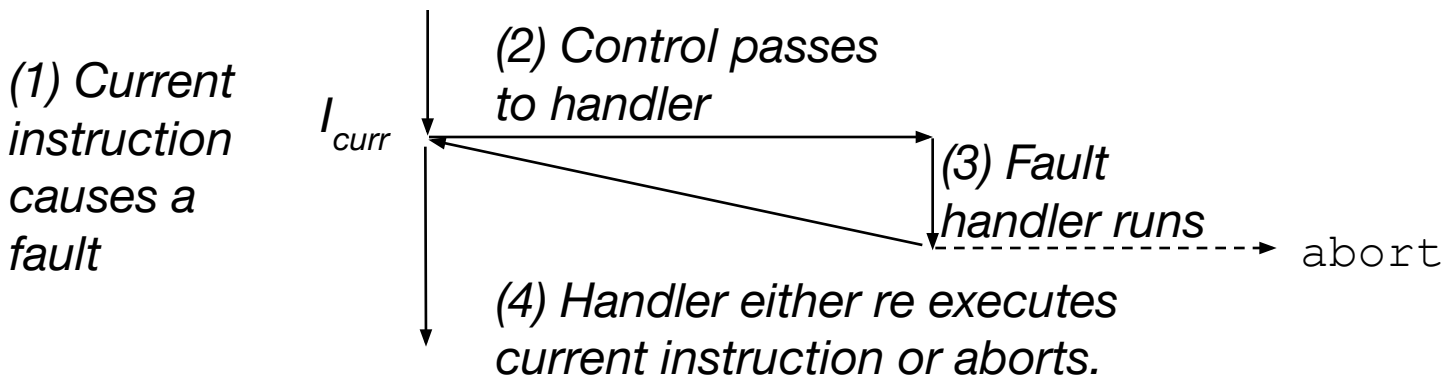


Faults

- Result in (possibly) correctable error conditions
  - If handler can correct condition, current instruction is retried
  - Otherwise program is aborted
- Examples:
  - Page fault: when memory needed by program has been swapped to disk
  - Protection fault: when program references unowned memory

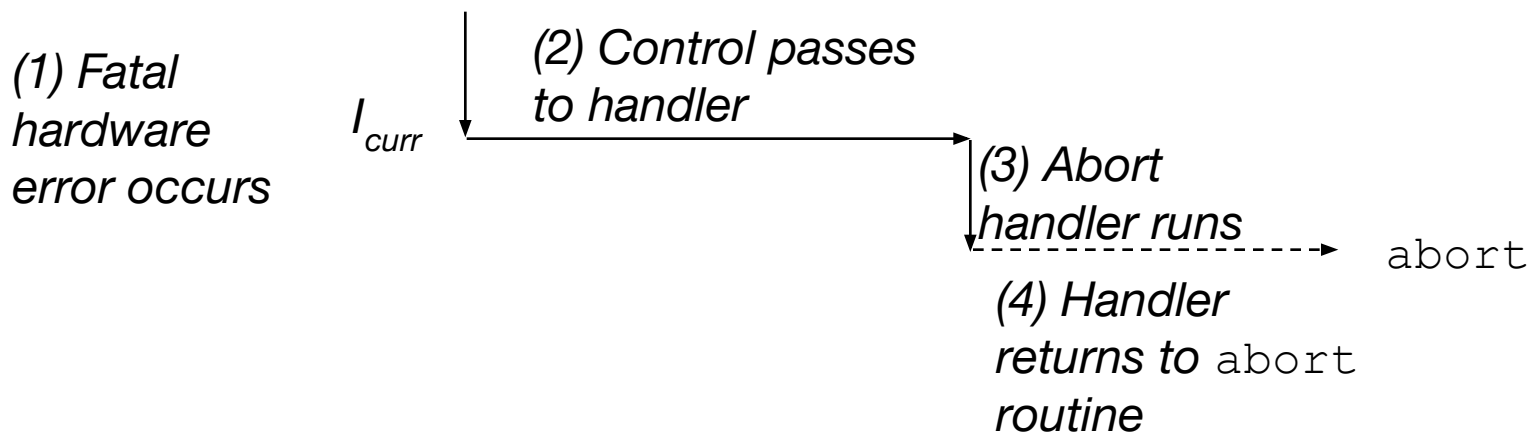
Fault Handling

- Current instruction generates a fault
- CPU identifies fault, assigns ID
- Uses ID to find handler in the exception table
- Invokes handler
- If handler corrects fault, handler returns to instruction that caused it
- Else handler invokes abort routine



Aborts

- Result from unrecoverable errors
  - Bad memory
  - CPU errors
- Same as a fault, but never returns, just aborts the process



Exceptions on Intel Processors

- Exception table has 256 entries
- Exceptions 0…31 are hardware defined (interrupts and faults)
- A trap is performed by the instruction int
- Form: int n where n is a constant between 0 and 255
- To return from an exception handler, use the special iret (instead of ret)
- The iret pops the FLAGS register before returning with the return address

Processes (Section 8.2)

- Are an abstraction for a running program
- Idea: Abstraction presents two illusions to a program
  - Exclusive use of the CPU: program instructions are executed without interruption
  - Exclusive use of the memory: program code and data are only objects in memory
- Definition: A processes is an instance of a program in execution

Processes

- A process defines the context in which the program runs:
  - Program code and data
  - Logical control flow of the program
  - Private address space
  - Contents of general purpose registers
  - Program counter

Processes

- Each time a program is run, the OS
  - Creates a new context
  - Loads the program code and initial data into context
  - Initializes context to begin execution of program
- Information about a process is stored in a **Process Control Block (PCB)**
  - Allocated by the OS when a process is created
  - Is only accessible to the OS
  - Used to store parts of the process context

Logical Control Flow

Logical Control Flow

- If we used a debugger to step through our program, we could trace the contents of our IP register (program counter or PC)
- Corresponds to addresses of instructions contained in our .exe file
  - Also any other objects linked to program
- This sequence is the logical control flow

Logical Control Flow

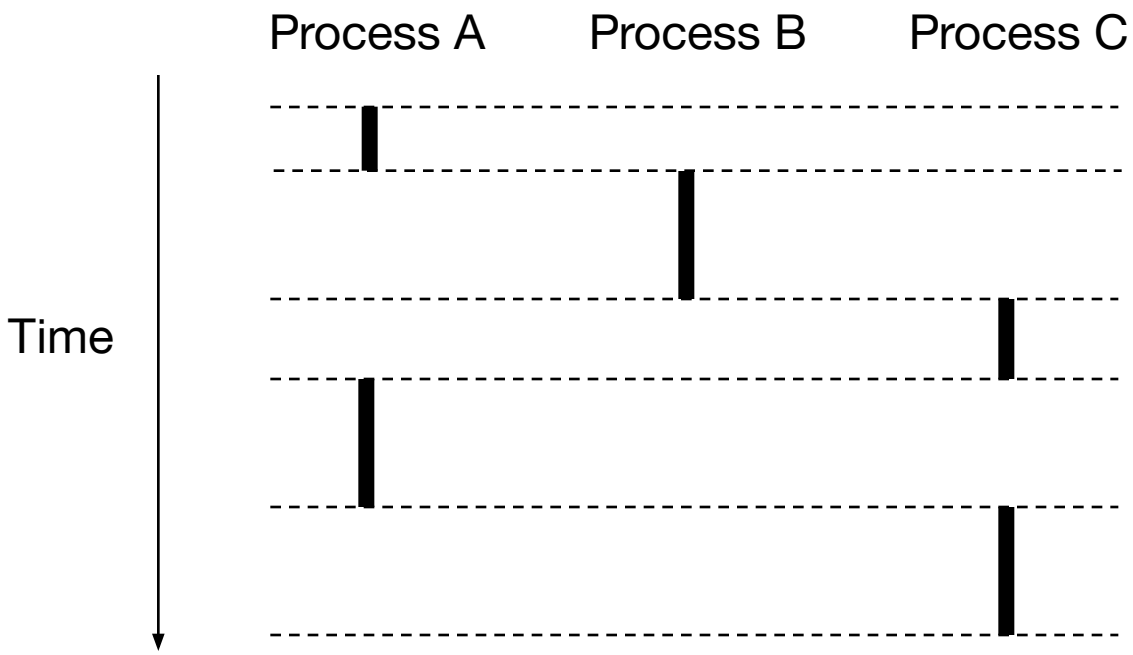
- This sequence of PC values is known as the logical control flow
- We can divide a **physical** control flow into **logical** control flows
- Each logical flow is independent of any other flow
  - Different processes do not affect the states of any other processes

Logical Control Flow

- Concurrent process: any process whose logical flow overlaps with another flow
- The notion of processes taking turns is called multitasking
- Each time period that a process executes is called a time slice

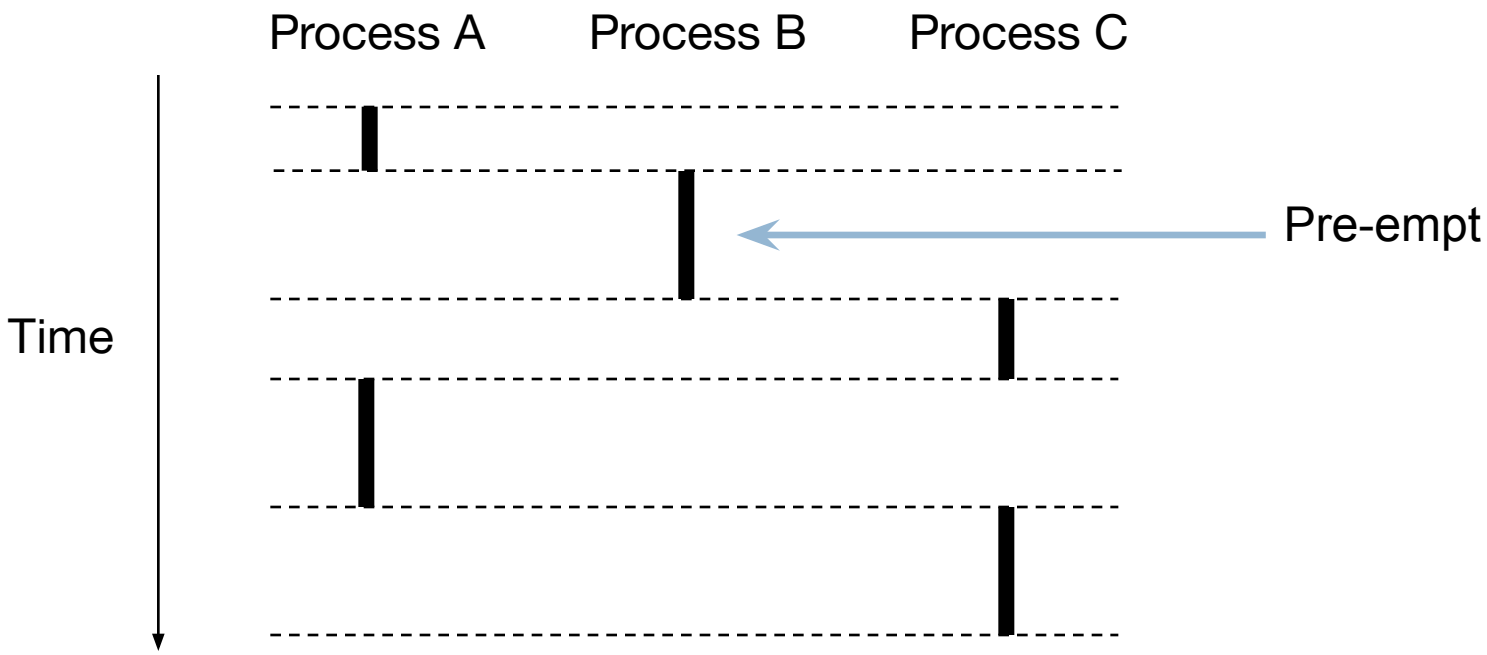
Logical Control Flow

- Each process executes a portion of its flow and then is pre-empted
- A & B and A & C run concurrently, but B and C do not



Logical Control Flow

- Each process executes a portion of its flow and then is pre-empted
- A & B and A & C run concurrently, but B and C do not

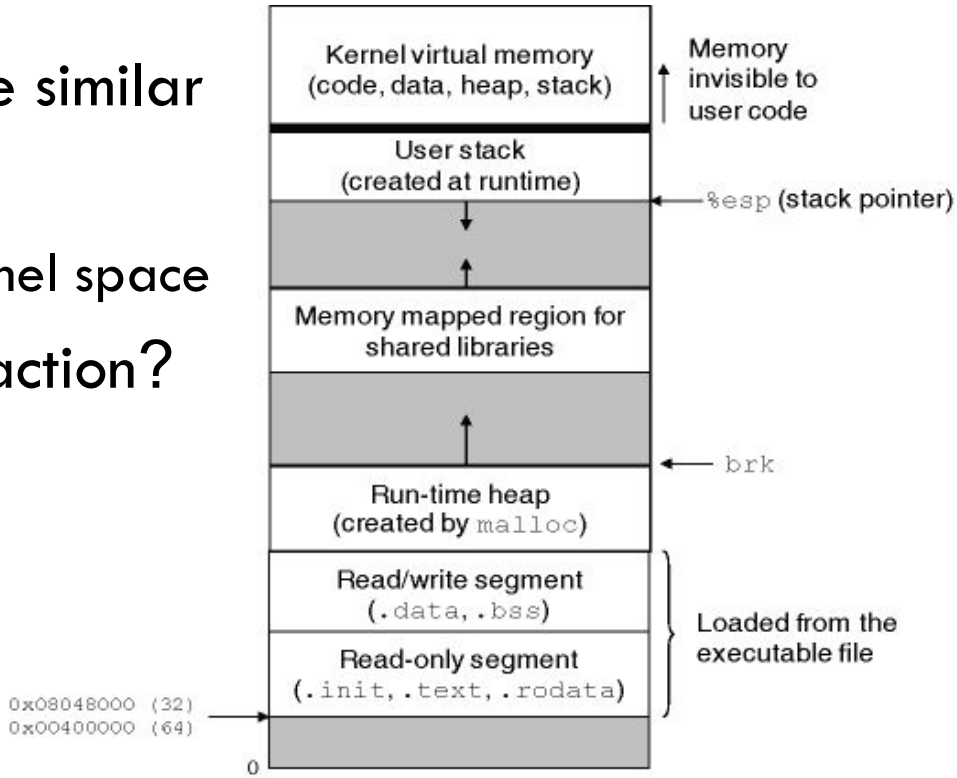


Logical Control Flow

- Notice that the idea of concurrency is independent of the number of cores
  - If two flows overlap, they are concurrent
    - Even if they are running on the same processor
- Sometimes we will find it useful to identify truly "concurrent" flows
  - Called parallel flows
    - Two flows running concurrently on different processor cores or processors
    - These flows are said to be running in parallel (have parallel execution)

Private Address Space

- Each process has illusion of exclusive use of system's address space
- In actuality, a process provides each program with its own private address space
- Private address spaces have similar layouts:
  - Code, data, heap, stack, kernel space
- Why do we need this abstraction?
  - To multitask



User and Kernel Mode

- Private address spaces must remain private!
- Need to restrict user capabilities
- Extra **mode bit** in a control register usually characterizes privileges of processor
- The mode bit sets the process in **user** or **kernel** mode

User and Kernel Mode

- Cannot do the following in user mode:
  - Halt processor
  - Change mode bit
  - Initiate I/O operation
  - Directly reference code in kernel area of address space

- All programs initially begin in user mode and use exceptions to pass into kernel mode

Multitasking

- In the past
  - Computer ran a program, waited until program completed, then ran the next one
- Today
  - We want many programs to run at once
  - No program should interfere with another
  - Programmers should not have to worry what other programs are doing

Multitasking

- Each program has its own logical flow, runs as if it is the only program on the computer
- Physically, many programs take turns running on the CPU
- The OS switches between the logical flows
- Creates the illusion that all programs are running at once
- In reality, the processes run concurrently

Multitasking

Key Ideas:

- To switch processes we simply switch contexts
  - Recall, context consists of address space, registers, flags, PC
- Processes are protected from each other because
  - Processes can only manipulate data inside their context
  - Processes may not manipulate data of another process (context)
  - Processes may not manipulate data of the OS
  - OS also runs in its own context
- A process need not worry about other processes because it is "alone"
- Question: when are processes switched in/out ?

Context Switching

- Recall: in normal control flow the program executes one instructions after the other
- Stopping a control flow is exceptional
- Thus, context switches occur only if an exception occurs
  - Trap
  - Interrupt
  - Fault

Context Switching

- What if no exceptions occur?
  - Computer has a timer chip
  - Generates a “timer” exception on fixed intervals
    - Between 10 to100ms
  - Exceptions are guaranteed to occur at regular intervals

Context Switching

- Recall: During an exception
  - Operating system takes control
  - Services the exception
  - Selects a process (could be the current one)
  - Returns flow control to that process
- The exception handler
  - **Switches out current process context**
  - **Switches in OS context**
  - Handles exception (OS code)
  - Selects a process to run (could be the same one)
  - **Switches out OS context**
  - **Switches in the process's context**
- Switch out/in is called a context switch

Context Switch

- Involves 2 contexts: the out-going and the in-coming
- Does the following:
  - Save CPU context (state) of current process
    - Stores all registers (FLAGS and PC included) into PCB
    - Essentially a sequence of mov instructions
  - Determine PCB of process/OS to be switched in
  - Switch memory address spaces (from the PCB)
  - Load CPU context (state) from current PCB
    - Load all registers FLAGS and PC from PCB
  - Run the switched process
- These steps are performed twice for a single exception

Context Switching

- Decision of kernel to pre-empt a process is called **scheduling**
- Handled by part of the kernel called the **scheduler**
- Question: Why would a kernel pre-empt a process in the first place?

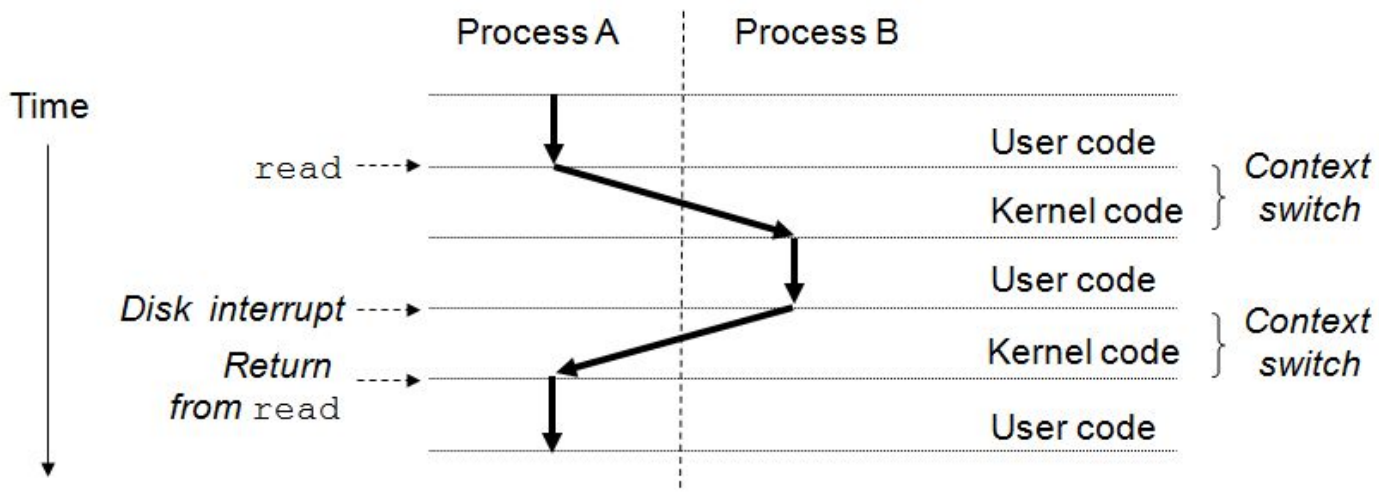
Context Switching

- Example:
  - Read call in process A requires disk access
  - Next instruction must be stalled until information is retrieved
  - Kernel decides to switch context for efficiency

- Programs can opt to sleep (request for context to be switched)
- Gets around any scheduling errors

User and Kernel Modes Revisited

- When an exception occurs
  - Control is passed to handler
  - CPU is placed in kernel mode
  - The handler now has access to OS memory, including PCBS
  - Can manipulate OS memory, perform context switch
  - When handler returns to process, CPU is placed back in user mode



Physical and Virtual Addressing

Section 9.1-9.6 (except 9.4)

Memory Sharing

- Process in a system share CPU and memory
- Poses some challenges:
  - Threshold in # of process, above which some may not be able to run
  - Memory is vulnerable to corruption
    - Process could write to another process's memory
    - Cause the process to fail in some odd fashion unrelated to the program logic
- Solution: Modern systems provide an abstraction of main memory known as **virtual memory**

Virtual Memory

- Interaction of hardware exceptions, hardware address translation, main memory, disk files, and kernel software
- Virtual memory provides three important capabilities:
  - Treats memory as a cache for an address space store on disk
  - Simplifies memory management by providing each process with a uniform address space
  - Protects address space of each process from corruption by other processes

Virtual Memory

Reasons to study virtual memory

- Virtual memory is central:
  - Encompasses all levels of computer system
  - Help you better understand system in general
- Virtual memory is powerful
  - Gives applications capabilities to create and destroy chunks of memory, map chunks of memory to portions of disk files, and share memory with other processes
  - Reading/writing virtual memory can modify contents of disk
  - Can load contents into memory without any explicit copying

Virtual memory

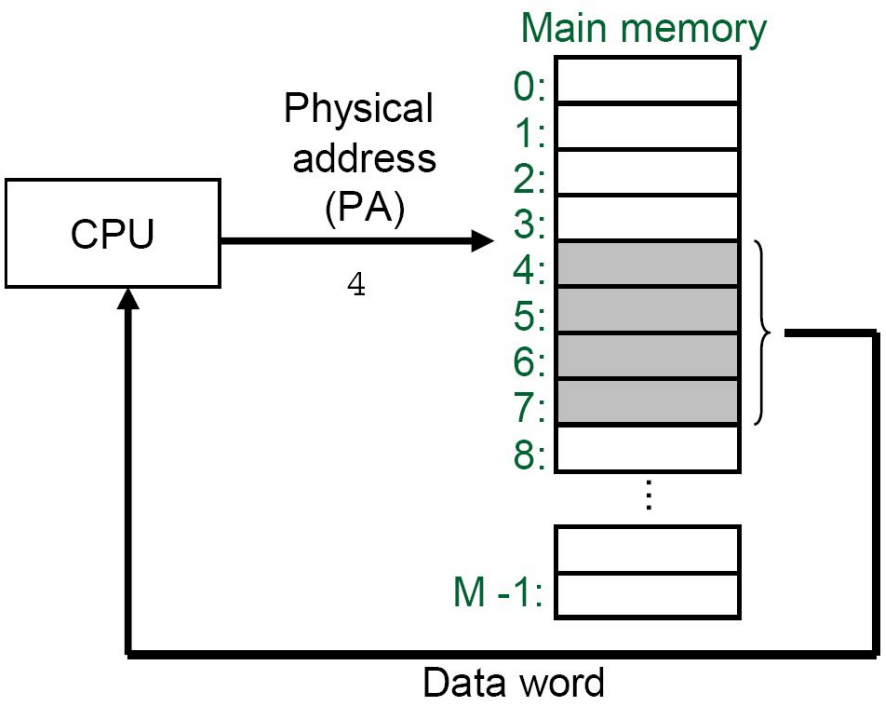
- Virtual memory is dangerous
  - If used improperly, applications can suffer from perplexing and insidious memory related bugs
    - Crash minutes or hours after problem occurred in the code
    - Run to completion with incorrect results

Physical Addressing (Sect. 9.1)

- Recall: computer main memory is an array of byte-sized cells (CPU point of view)
- Each byte has a unique physical address (PA)
  - Most natural way for CPU to access memory is to use physical addresses
- Called physical addressing
  - Address corresponds to same physical location in memory
  - The address is from the physical address space
- This is the easiest and most natural approach

Physical Addressing

- Example: a load instruction that reads the word at address 4
- CPU generates physical address
- Sends it over the bus
- Memory fetches four-byte word starting at address 4



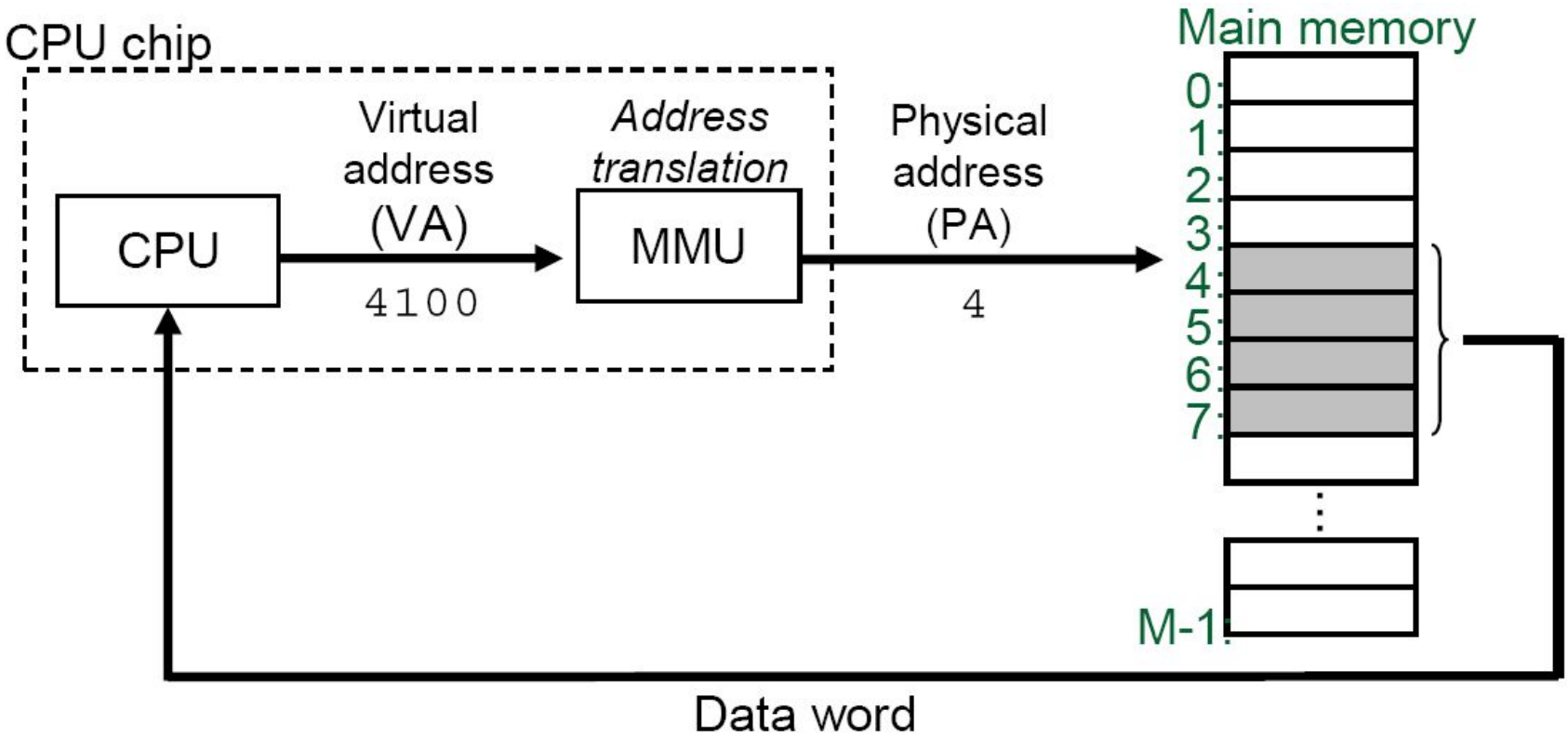
Virtual Addressing

- Modern computers have CPU access main memory by generating a virtual address (VA)
- VA is then converted to PA before being sent to memory
- The task of converting a VA to a PA is called **address translation**

Virtual Addressing

- Requires close cooperation between CPU hardware and the OS
- Translation performed by hardware on CPU called the **memory management unit** (MMU)
- Addresses are translated on the fly using a look-up table stored in main memory
  - Contents managed by the OS

VA Example


CPU chip
CPU
Virtual address (VA)
4100
Address translation
MMU
Physical address (PA)
4
Main memory
0:
1:
2:
3:
4:
5:
6:
7:
M-1:
Data word

Virtual Addressing

- Called virtual addressing
  - Each address is virtual from a virtual address space
  - Translated by MMU
  - Process thinks it is reading/writing at the specific address
  - In reality, the read/write occurs at MMU translated address
  - Allows system to control where the processes read/write memory without process involvement

Address Spaces (Sect. 9.2)

- An **address space** is an ordered set of nonnegative integer addresses {0, 1, 2, …}
- Linear address space: If the integers are consecutive
  - e.*g*. physical main memory addresses form a linear address space
  - We will always assume linear address spaces

Address Spaces

- CPU generates virtual address space of $N = 2^n$ address
  - $\{0, 1, 2, \ldots, N - 1\}$
- Address space size is characterized by the number of bits needed to represent largest address
  - e.g. virtual address space with $N = 2^n$ addresses is called a n-bit address space
  - Modern systems use either 32 or 64-bit virtual address spaces
- A system also has a physical address space that corresponds to M bytes of physical memory
  - $\{0, 1, 2, \ldots, M - 1\}$

Address Spaces

- Address space concept is important because it makes a clean distinction between data objects (bytes) and their attributes (addresses)
- This allows each byte to have
  - Virtual address from the virtual address space
  - Physical address from the physical address space

VM and Caching (Sect. 9.3)

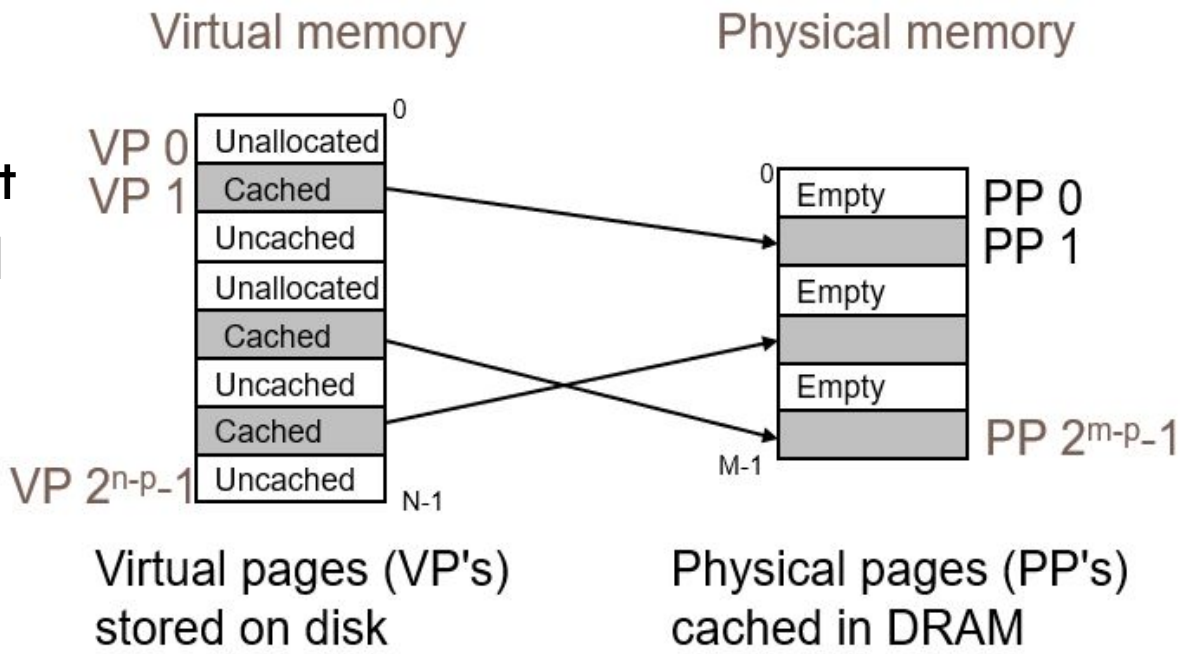
- VM is organized as an array of N contiguous byte-sized cells stored on disk (hard drive)
- Each byte has a unique address (index into array)
- The contents of the array on disk are cached in main memory
- The data on disk is partitioned into blocks that serve as the transfer units between the disk and the main memory
  - VM is partitioned into fixed-sized blocks called **virtual pages** (VP)
  - Each virtual page is $P = 2^p$ bytes in size
  - Physical memory is partitioned into **physical pages** (PP)
  - Physical pages are also P bytes in size

VM and Caching

- At any point in time, the set of VP is partitioned into three disjoint subsets
  - Unallocated: Pages that have not yet been allocated (or created) by the VM system
    - Contains no data and do not occupy any disk space
  - Cached: Allocated pages that are currently cached in physical memory
    - Page contain data
    - Page is assigned to a frame (physical page)
  - Uncached: Allocated pages that are not cached in physical memory
    - Page contains data
    - Page is not assigned to a frame (physical page)
    - Page is stored on disk

Example

- 8 virtual pages
- 0 and 3 have not been allocated yet
- 1, 4, 6 are cached in physical memory
- 2, 5, 7 are allocated but are not currently cached



Key Ideas

- Physical frames (pages) hold virtual pages
- A virtual page can be placed in any frame
- Not all pages or frames are used
- **A process must allocate a page before using it**
  - Requests a page of memory from the OS
  - OS creates a new virtual page
  - Finds a free frame (physical page)
  - Assigns page to the frame
  - Returns confirmation to process

VM and Caching

- When a process performs a read/write
  - CPU sends virtual address to MMU
  - MMU determines page that contains the address
- MMU checks if page is unallocated, cached, or uncached
- Unallocated causes an exception, program is terminated
- Uncached causes an exception, invokes the VM:
  - Locates a free frame (physical page)
  - Loads page from HD into frame
  - Adds mapping updates to MMU
  - Returns to process, attempts memory access again

Paging Tables

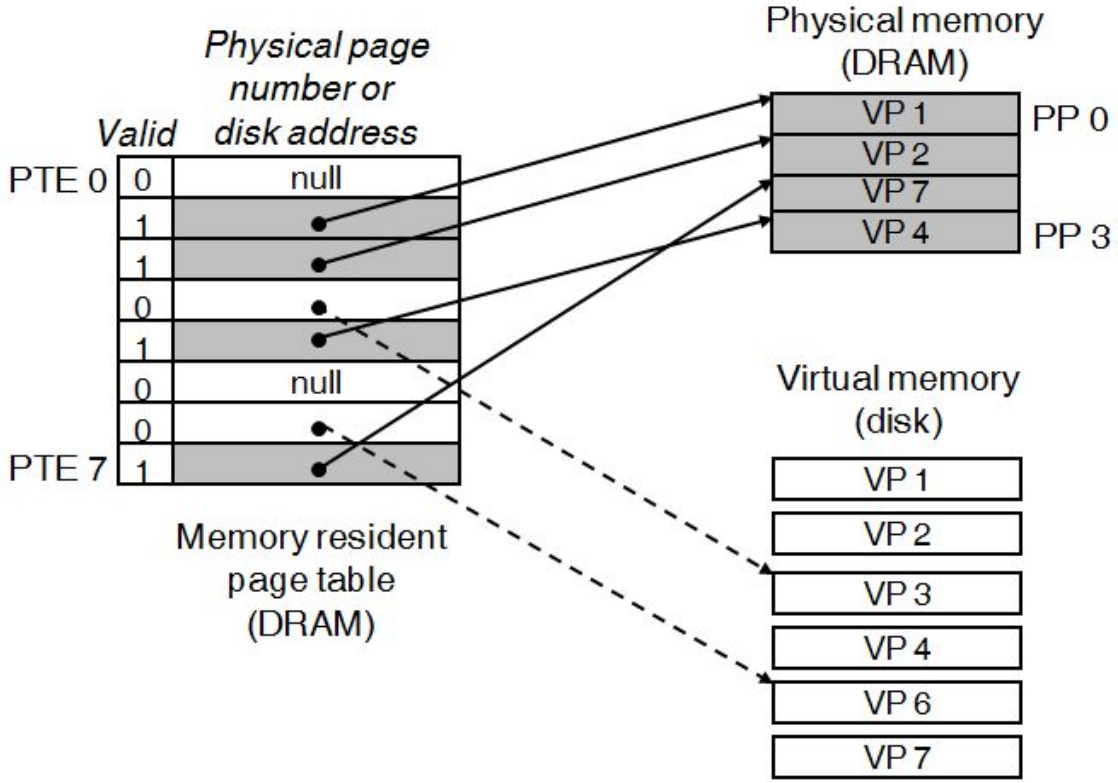
- Task: The MMU has to translate addresses for each running process
- Basic approach: for each virtual address determine
  - Which virtual page corresponds to the address
  - Whether virtual page is allocated and cached
  - Determine location of frame containing the page

Paging Tables

- To perform the translation the MMU uses page tables
  - Each process has an associated table
  - The tables are stored in physical memory
  - Contain page table entries (PTE)

Example

- Page table for 8 VP
- 1, 2, 4, and 7 are cached
- 0 and 5 not allocated
- 3 and 6 are not cached
- Note: any physical page can contain any virtual page



Paging Tables

- One PTE per page in the virtual address
- Size of table is size of virtual address space / size of page
- Example:
  - Suppose $N = 65536 = 2^{16}$
  - Each page is of size $P = 4096 = 2^{12}$
  - # entries $= N/P = 2^4 = 16$
- Each PTE contains:
  - Valid bit
  - Address entry
- Valid bit: denotes if the entry is cached or not

Paging Tables

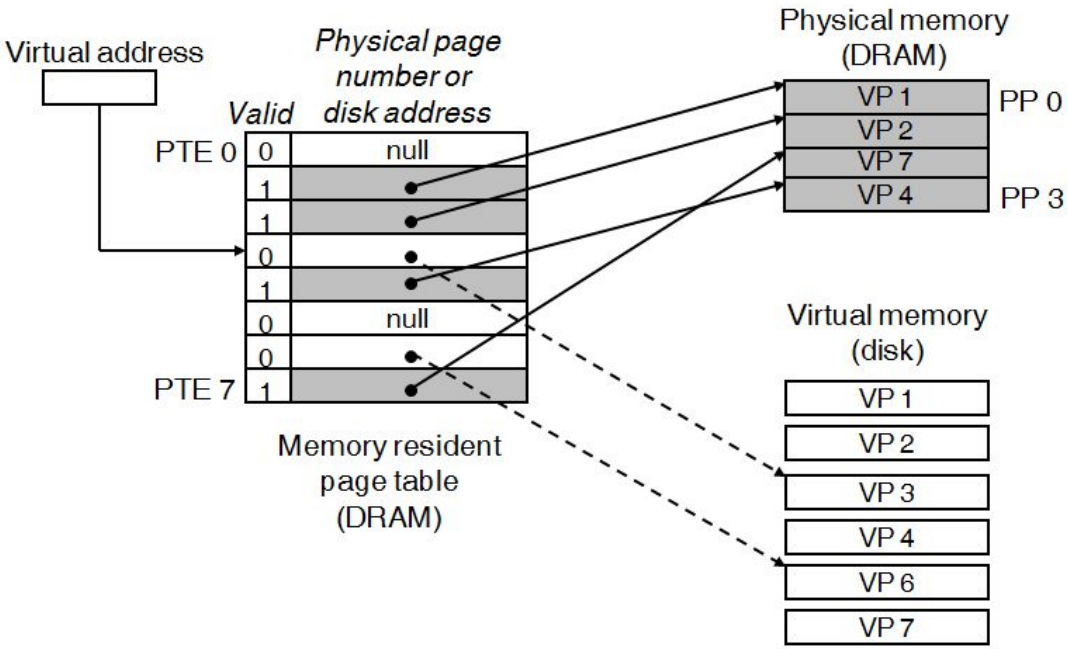
- Address denotes one of three things
  - 1: address contains location of frame (physical page)
  - 0:
    - Address contains location of page on disk
    - Or null address means the page is unallocated
- Hence: a page is
  - **Unallocated** if valid = 0, and address = 0
  - **Not cached** if valid = 0, and address != 0
  - **Cached** if valid = 1

Processes and Memory Access

- CPU sends address to MMU
- MMU checks if page is in memory
- If page is cached
  - Called a page hit
  - MMU translates virtual address into physical address
  - Sends it to the memory system to read/write data
- If not
  - Called a page miss
  - Similar to cache hit/miss
  - Page is loaded
  - Current instruction retried

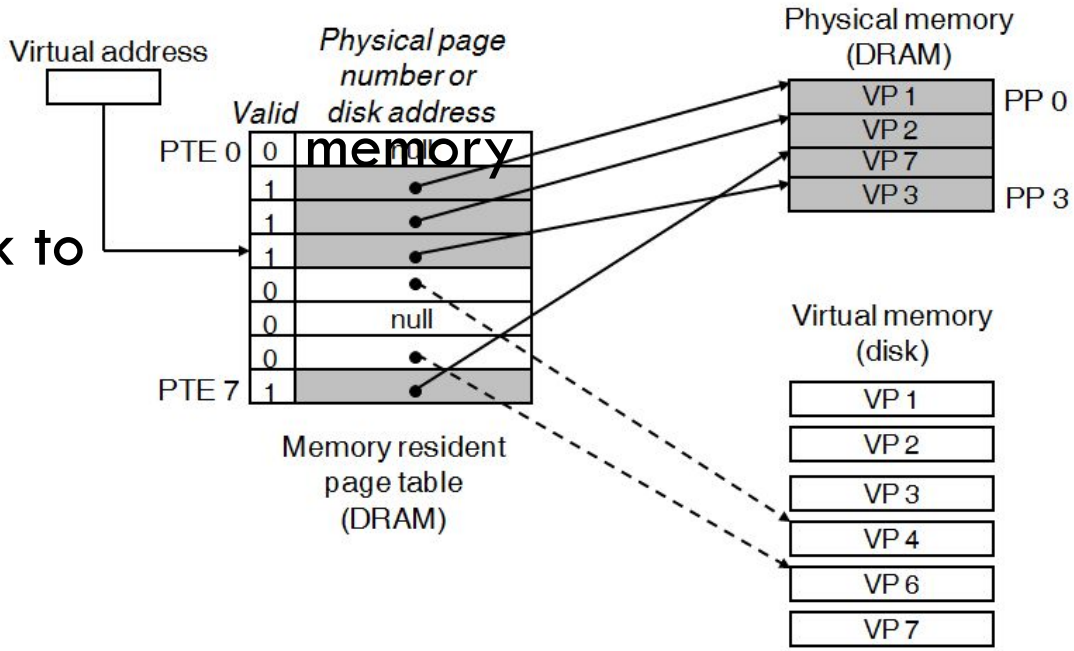
VM Page Fault (Before)

- CPU references VP 3
- Hardware reads PTE 3
- Infers from valid bit that VP 3 is not cached
- Page fault exception occurs



VM Page Fault (After)

- Exception handler selects a victim page
- In this case VP 4 stored in PP 3
- If VP 4 has been modified kernel copies it back to disk
- Kernel modifies PTE to reflect VP 4 is no longer in memory
- Kernel copies VP 3 from disk to PP 3 in memory
- PTE 3 is updated
- Faulting instruction restarted



Page Fault Summary

- MMU issues an exception
- Invokes page fault handler
- The handler
  - Checks to if the page is allocated
  - If the page is not allocated, process is aborted
  - Otherwise check for a free frame
  - If a free frame is available, use it
  - If not
    - Selects a victim page to be evicted
    - If the page has been modified, write it to HD
    - Update page table entry
    - Use resulting free frame
  - Load required page from HD into free frame
  - Update process' page table

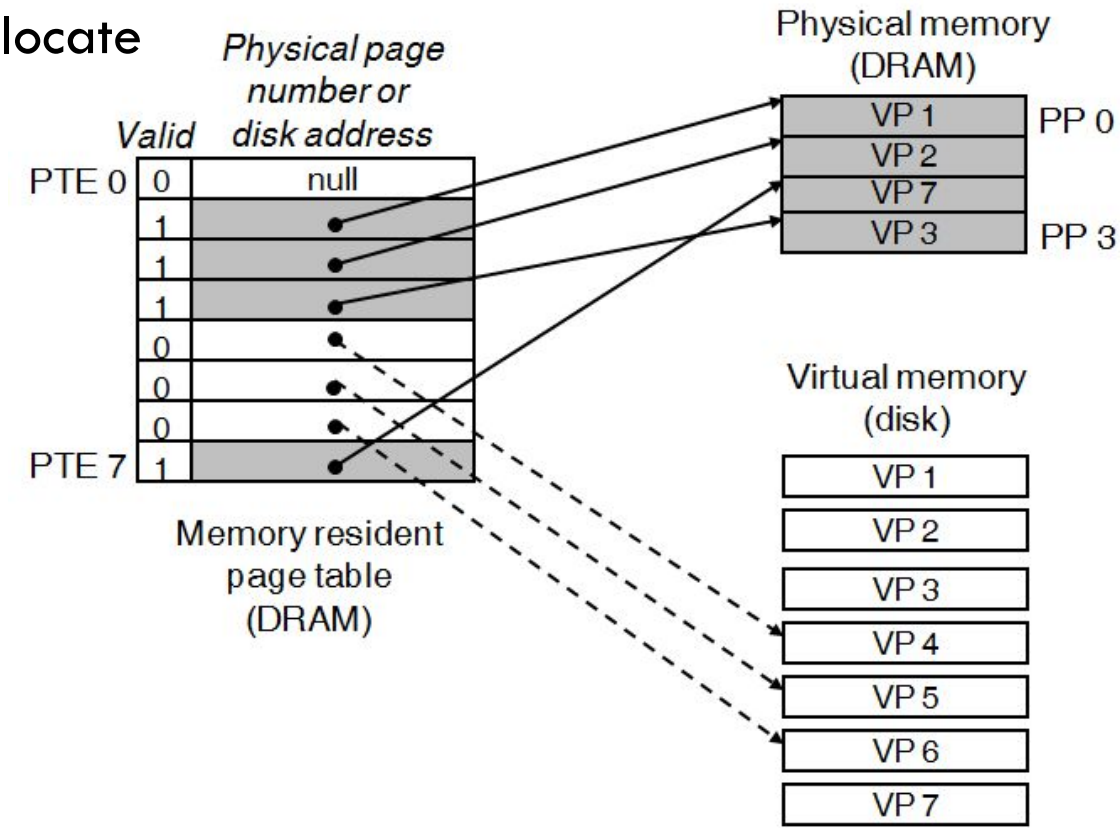
- Handler returns to faulting process
- Restarts same instruction that created fault
- Process proceeds as if nothing has happened

Page Allocation

- Pages are allocated under three conditions
  - When a process is created
    - Enough pages are allocated to hold code, data, stack, and small heap
  - When a process needs more heap memory
    - Process uses system call to request pages from OS
    - OS responds by
      - Allocating pages for the heap
      - Returns to the process # of pages that were added
  - When process needs more stack
    - OS automatically detects when a process needs more stack
    - Allocates more pages to the stack

Page Allocation

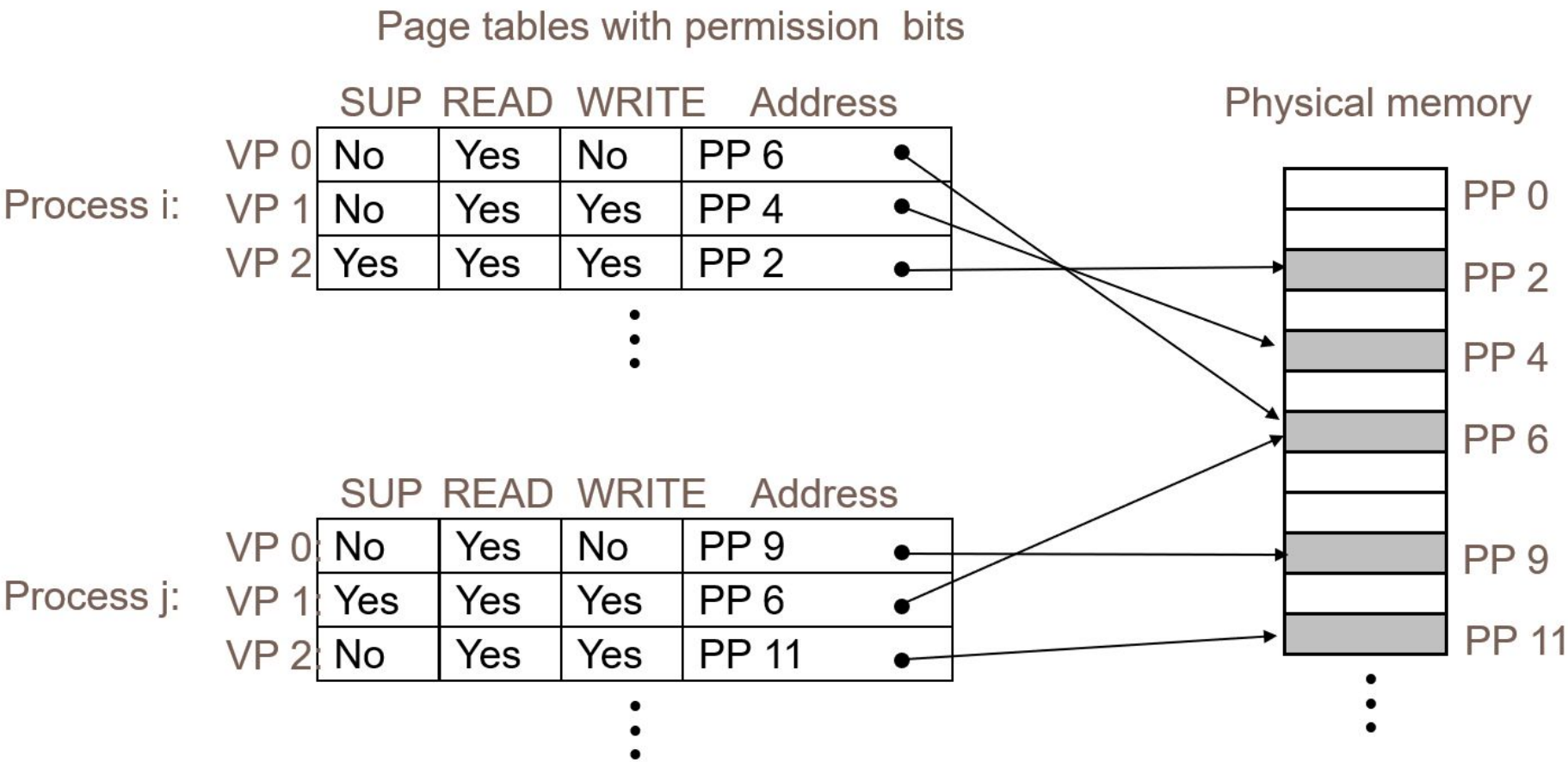
- When pages are allocated:
  - OS modifies corresponding page table entries
  - Page is created on disk
  - OS may or may not allocate
- VP to PP



Memory Protection (Section 9.5)

- A useful feature of VM is memory protection
- Basic idea:
  - Process only sees its own memory
  - Hence it can only modify its own memory
- Problem: process also "sees" kernel memory in private address space
- Solution: Each PTE contains additional permission bits
  - Read bit, write bit, execute bit, supervisor bit

Memory Protection



Memory Protection

- Suppose the MMU not only checks if the page is resident but also
  - If read operation, whether read bit is set
  - If write operation, whether write bit is set
  - If CPU is in user mode, whether supervisor bit is not set
- If any of these test fail, MMU generates a **general protection fault**

Memory Protection

- Important because
  - Kernel sets permissions on memory accessible by the process
  - This prevents process from reading, writing or executing any memory that is dangerous
- Thus, VM serves as a natural protection system

Aside: VM and Caching

- Note that valid bit in PTE denotes whether information is cached or not
- Page hits and page misses analogous to cache hits and cache misses from Chapter 6
- Waiting until the last minute to swap a page from DRAM to disk is analogous to write-back caches

VM and Locality

- In the previous examples, we saw how VM consisted of pages from both DRAM and disk
- Given disk access times, why isn't VM terribly inefficient?
- Answer: Locality to the rescue!

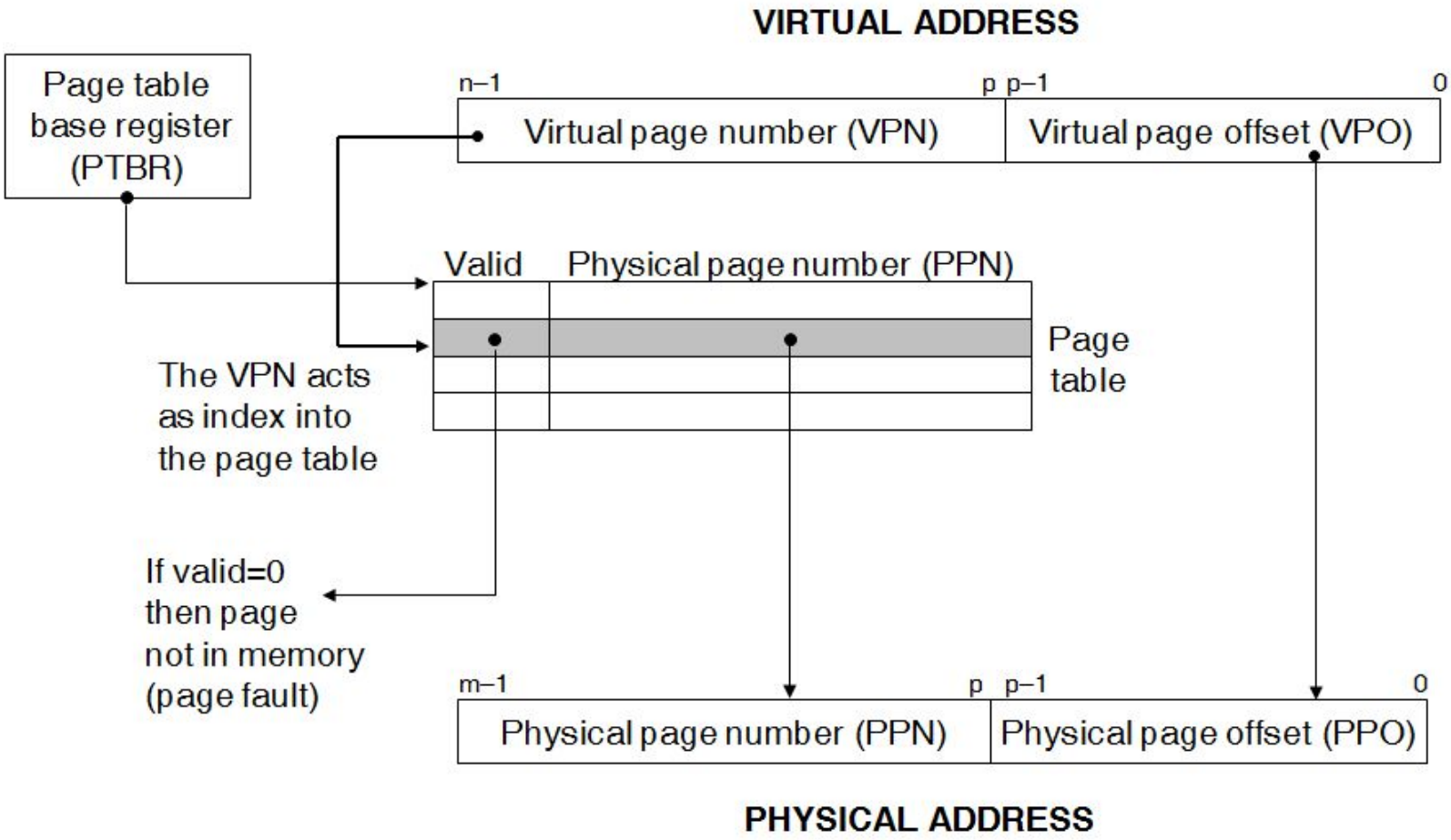
VM and Locality

- Total # of pages probably exceeds total size of physical memory
- BUT: Locality promises at any point in time they will tend to work on smaller set of active pages (working set)
- See memory mountain

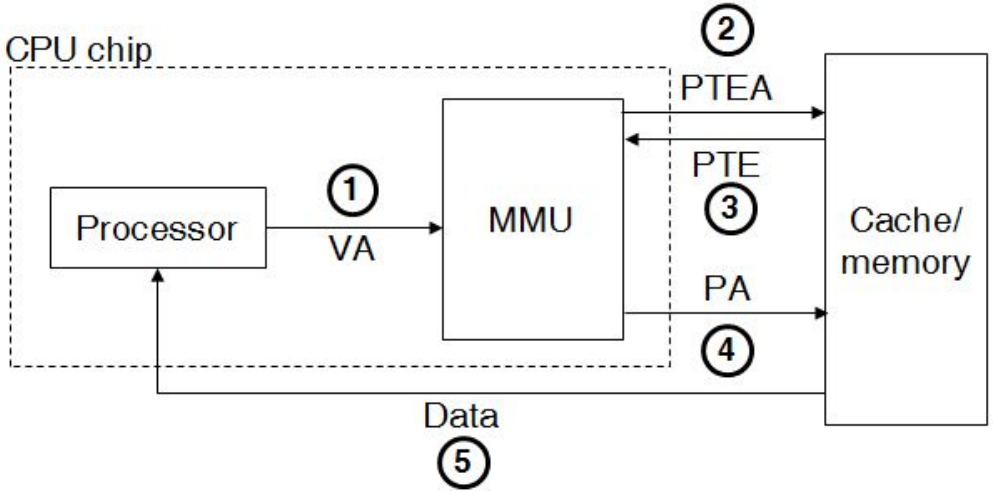
Address Translation (Sect. 9.6)

- Basic translation: virtual to physical
- For a given address the MMU must
  - Locate corresponding PTE
  - Determine location of frame that contains the page
  - Compute offset into frame
  - This is the resulting address
- Process is similar to cache
- Virtual address is divided into two parts
  - Virtual page number: VPN
  - Virtual page offset: VPO
- MMU uses the VPN to locate PTE
- Entry in PTE is: physical frame (or page) number
- MMU composes PPN and VPO to get physical address

Address Translation

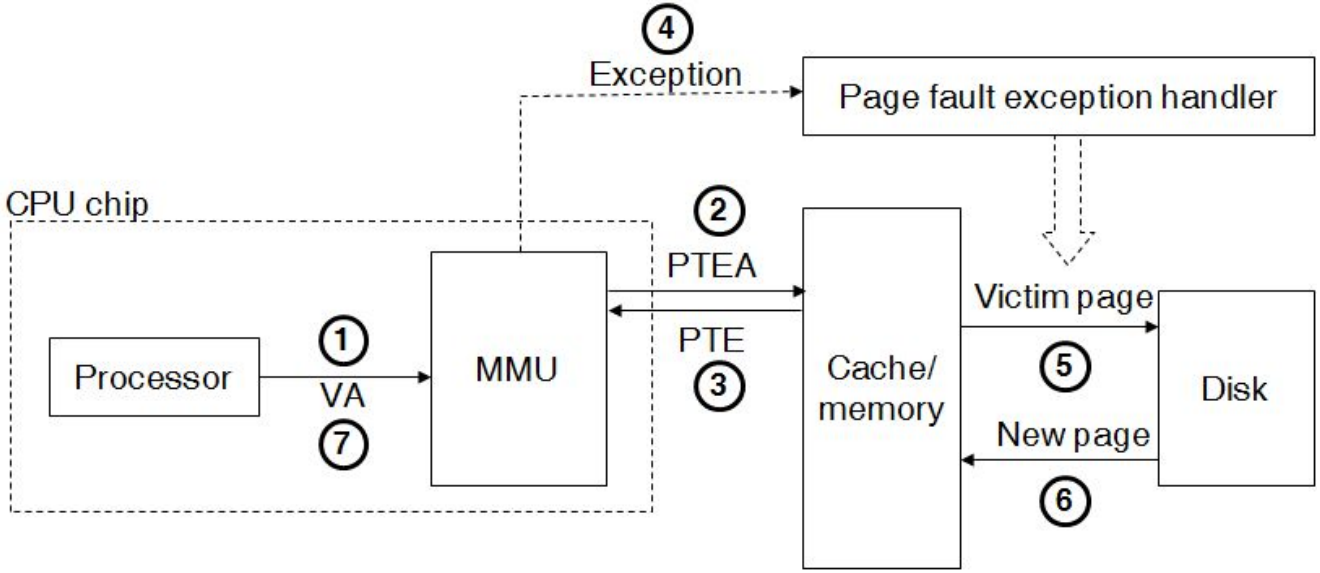


Address Translation

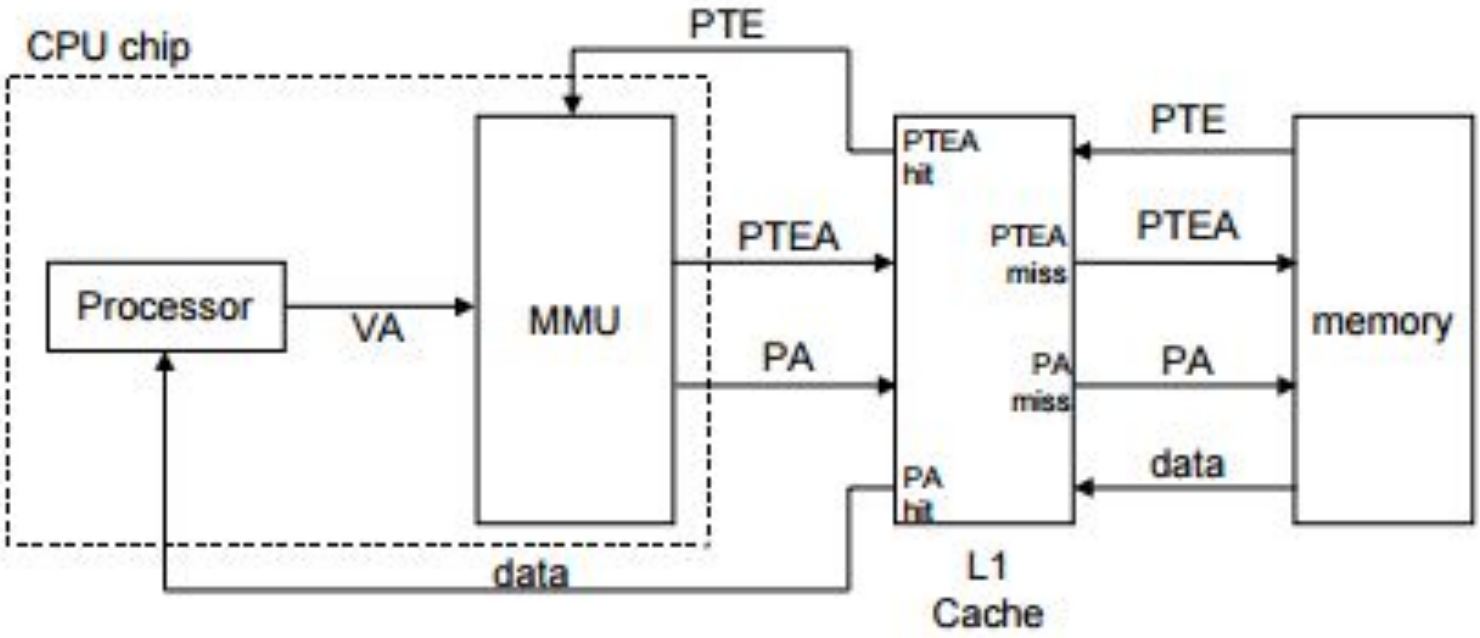


Page Hit

Page Miss

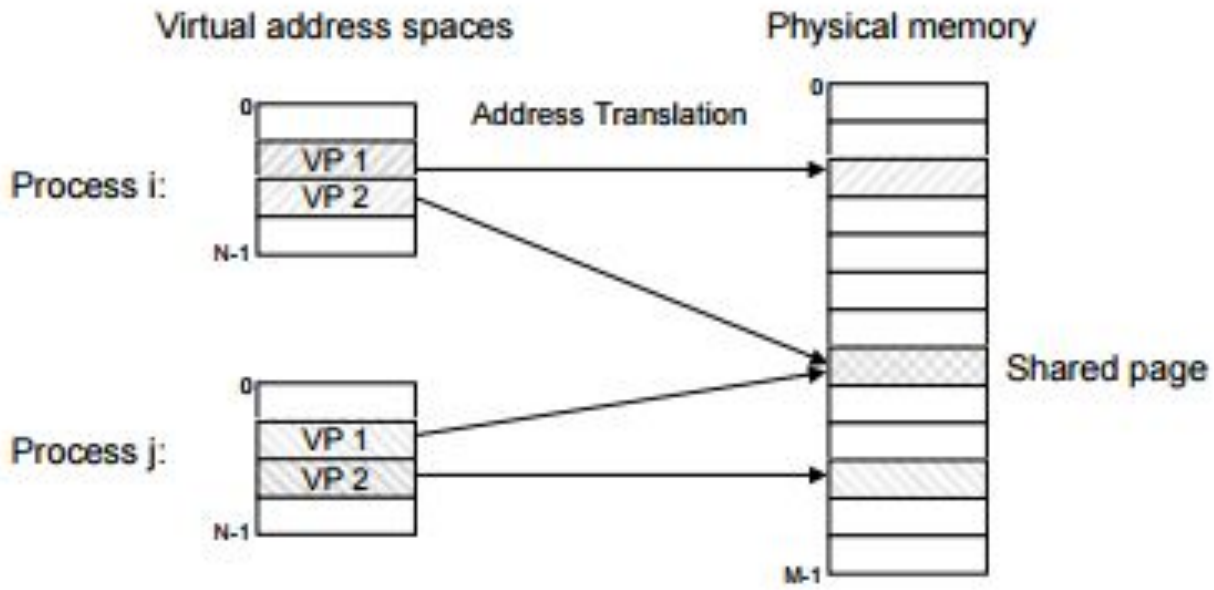


Address Translation



Address Translation

- PTEs are kept separate for processes, even if they share the same physical memory



- PTE's may also be multi level (PTE pointing to a PT)

TLB

- Problem: this process requires accessing more memory (to read PTE)
- Solution: use a special cache called **Translation Lookaside Buffer**
- Most programs have good locality
- *i.e.* accesses are spatially and temporally localized
- Many accesses will occur within the same set of virtual pages and hence within the same set of physical frames

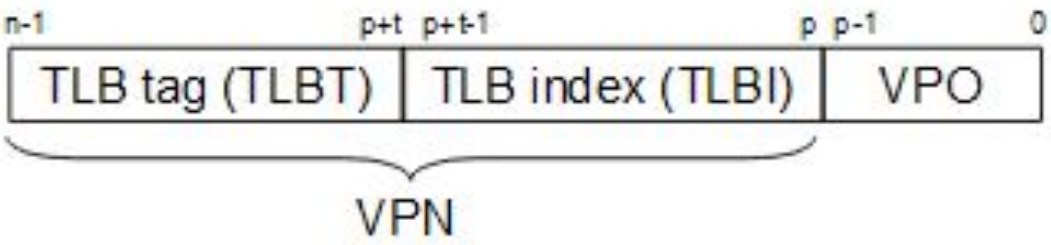
TLB

Idea:

- Cache PTEs of recently accessed virtual pages in a cache
  - Located in the MMU
  - Called TLB
- Instead of looking up PTE in page table first check TLB
- If PTE is in TLB, use it
- Else look in page table
  - Load TLB with new PTE (may need to evict another one)
- TLBs are efficient (95% hit rates)

TLBs

- Translation Lookaside Buffer is used as a cache for page table entries
- Indexing into a TLB is done through tag and index bits
- Very similar to caching in chapter 6



Putting it all together

- You should now be able to explain the steps involved in a memory read beginning with an arbitrary virtual address A, ending with an arbitrary DRAM supercell address (i,j)

- You should be able to describe the functions of:
  - MMU
  - Buses
  - I/O controller
  - Mem/Disk Controllers
  - Caches (On-chip and off), (Write-through/write-back)